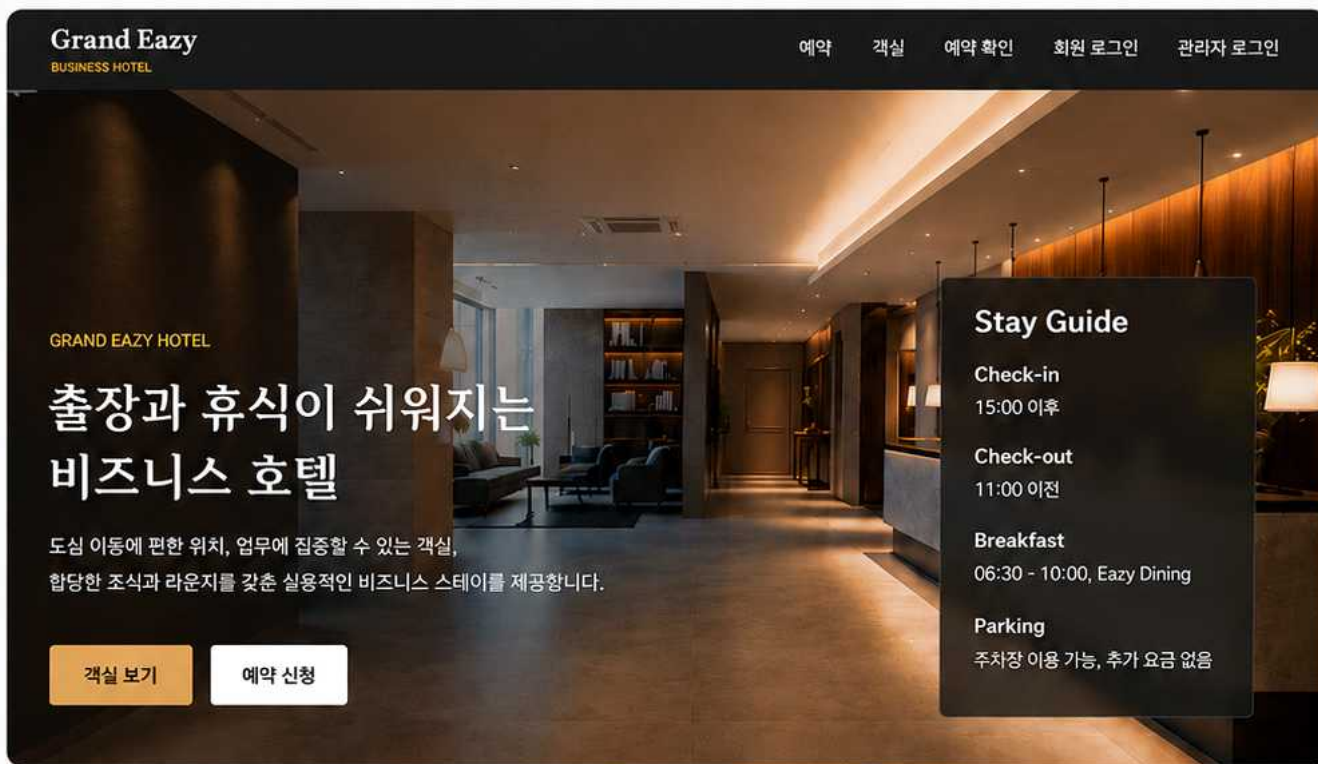


모의해킹 결과보고서

Penetration Test Report



Eazyhotel 호텔 예약 웹 서비스 보안 취약점 진단 및
보안 관제 시스템 모의해킹 수행 결과



프로젝트 명	Eazyhotel (Hotel Reservation Security Monitoring System, HRSMS)
대상 시스템	Eazyhotel 호텔 예약 웹 서비스
수행팀	2지(Easy)하조
테스트 및 작성	2026.06.05 ~ 2026.06.11

목 차

1. 모의해킹 수행 정보	3
1.1. 개요	3
1.2. 대상	3
1.3. 수행 기간	3
1.4. 수행 인력	3
2. 모의해킹 결과	4
2.1. 총평	4
3.1. 인증 관련 취약점	5
3.1.1. SQL 인젝션 취약점 (인증 우회)	5
3.1.2. Open Redirection (오픈 리다이렉션)	8
3.1.3. 세션 고정(Session Fixation) 취약점	11
3.2. 회원 관리 관련 취약점	15
3.2.1. Stored XSS (저장형 크로스사이트 스크립팅)	15
3.2.2. 부적절한 인가(IDOR) 취약점 (예약 정보 노출 및 조작)	18
3.3. 시스템 접근 취약점	22
3.3.1. SSRF(Server-Side Request Forgery) 취약점	22

1. 모의해킹 수행 정보

1.1. 개요

본 프로젝트는 [Eazy hotel]의 호텔 페이지에 대한 모의 해킹을 수행한 후 발견된 취약점에 대한 대응 방안을 제시함으로써 차후 발생할 수 있는 침해 사고를 예방하고 정보 보호 수준을 향상할 수 있는 대책을 수립하는 데 기여하는 것을 목적으로 합니다.

1.2. 대상

본 프로젝트 점검 대상은 다음과 같습니다.

No	Platform	URL
1	WEB	https://eazyhotel.com

1.3. 수행 기간

2026 년 06 월 05 일 ~ 2026 년 06 월 11 일

1.4. 수행 인력

김혜미, 남재근, 홍기수

2. 모의해킹 결과

2.1. 총평

본 시스템에 대한 보안 진단 결과, 총 6건의 주요 취약점이 식별되었습니다. 진단 항목은 크게 '인증', '회원 관리', '시스템 접근' 영역으로 구분되며, 각 항목에서 발견된 기술적 결함들은 시스템의 전반적인 보안 무결성과 데이터 보호 체계에 중대한 위험 요소로 작용하고 있습니다.

주요 진단 결과:

- 인증 및 세션 제어 취약점 (3.1.1 ~ 3.1.3):**
SQL 인젝션을 통한 인증 우회 가능성이 확인되었으며, Open Redirection 및 세션 고정(Session Fixation) 취약점이 도출되었습니다. 이는 공격자가 정상적인 인증 절차를 회피하거나, 탈취된 세션 정보를 이용하여 타인으로 위장 접속할 수 있는 심각한 위험을 내포하고 있습니다.
- 회원 정보 관리 및 인가 제어 취약점 (3.2.1 ~ 3.2.2):**
저장형 XSS(Stored XSS) 취약점을 통해 악의적인 스크립트 실행 및 사용자 정보 탈취가 가능하며, 부적절한 인가(IDOR) 결함으로 인해 타인의 예약 정보를 열람하거나 임의로 조작할 수 있는 상황입니다. 이는 개인정보 보호 및 서비스 데이터 무결성에 치명적인 영향을 줄 수 있습니다.
- 시스템 접근 제어 취약점 (3.3.1):**
SSRF(Server-Side Request Forgery) 취약점이 식별되어, 외부에서 시스템 내부 자원으로 비인가 요청을 수행할 가능성이 확인되었습니다. 이를 통해 서버 내부 인프라에 대한 추가적인 공격 시도가 우려됩니다.

결론 및 제언:

현재 본 시스템은 인증 우회, 데이터 조작, 서버 내부 망 접근 등 웹 보안의 핵심 위협 요소들이 복합적으로 존재하는 상태입니다. 각 항목별 발견된 취약점은 단독으로도 시스템의 보안을 무력화할 수 있는 수준이므로, 개발 단계에서의 보안 코딩 적용 및 검증 로직 강화를 통해 발견된 모든 취약점에 대한 즉각적인 조치가 요구됩니다.

2.2 결과 요약

No	대상	취약점	위치
1	로그인 페이지	SQL 인젝션	http://eazyhotel.com/member_login.php
2.		Open Redirection	http://eazyhotel.com/member_login.php
3.		세션 고정	http://eazyhotel.com/member_login.php
4.	회원가입 페이지	Stored XSS	http://eazyhotel.com/member_register.php
5.	예약 상세 페이지	IDOR	http://eazyhotel.com/ member_reservation_detail.php
6.	메인 페이지	SSRF	https://eazyhotel.com/main.php

3. 취약점 상세 내용

3.1. 인증 관련 취약점

3.1.1. SQL 인젝션 취약점 (인증 우회)

구분	내용
위험	상 (High)
발견URL	http://eazyhotel.com/member_login.php
취약점 상세내용	로그인 페이지 아이디 필드에서 입력값 필터링이 미흡하여 SQL Injection 발생. 인증 과정을 조작하여 비밀번호 없이 로그인이 가능함.
재현 페이로드	' OR 1=1 LIMIT 1,1 #

취약점 상세 내용

- 사용자가 입력한 데이터를 데이터베이스 쿼리문에 전달할 때, 별도의 이스케이프(Escape) 처리나 매개변수화된 쿼리(Prepared Statement)를 사용하지 않고 그대로 실행하여 발생합니다.
- 공격자의 입력값인 ' OR 1=1 LIMIT 1,1 #이 쿼리에 결합되면서 WHERE 조건절이 항상 참(True)이 되도록 조작됩니다.
- 본 취약점은 단순한 인증 우회를 넘어, 공격자가 LIMIT [offset], [row_count] 구문을 조작하여 DB 내 특정 사용자 계정을 선택적으로 타깃팅하여 탈취할 수 있으므로 관리자 권한 탈취 및 개인정보 유출로 이어질 수 있는 매우 위험한 상태입니다.

재현 페이로드 및 절차

[조치 전] 취약점 코드 및 상태

사용자 입력값을 SQL 구문에 문자열로 직접 바인딩하여 `` OR '1'='1` 등의 구문 입력 시 패스워드를 몰라도 최고 권한자로 인증 우회가 가능하며, CSRF 토큰 및 패스워드 해시 검증이 모두 철회되어 있음.

```
// 1. 방어선 해제 (CSRF 검증 및 공백 가공 함수 무력화 상태)
$username = (string) ($_POST['username'] ?? '');
$password = (string) ($_POST['password'] ?? '');

// 2. SQL 쿼리문 직접 결합 (SQL Injection 발생 지점)
$sql = "SELECT id, username, password_hash, role, display_name FROM users WHERE username = '$username'";
$stmt = $pdo->query($sql);
$user = $stmt->fetch();

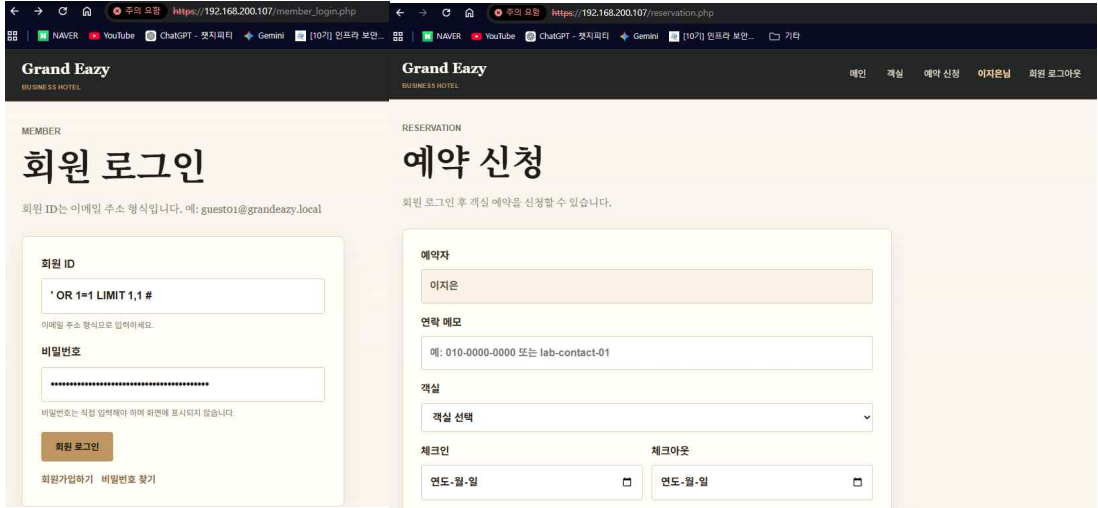
// 3. 비밀번호 검증 무력화 (결과 행만 존재하면 패스워드 대조 없이 무조건 통과)
if ($user) {
    $success = true;
    // ... 세션 등록 및 로그인 처리
}
```

모의해킹 절차

1. **데이터베이스 사용자 계정 상태 확인:** 내부 MariaDB(hotel_lab) 조회를 통해 users 테이블에 admin(0번 index), 이지은(1번 index), 김민준(2번 index) 순으로 계정이 등록되어 있음을 타킷으로 인지합니다.
2. **공격 시도 및 페이로드 대입:** 로그인 페이지 회원 ID 입력란에 비밀번호를 모르는 상태에서 변조된 페이로드(' OR 1=1 LIMIT 1,1 #)를 삽입한 후 로그인을 시도합니다.
3. **인증 우회 결과 확인:** LIMIT 1,1 구문에 의해 두 번째 레코드인 '이지은' 회원으로 인증이 우회되어 비밀번호 없이 예약 신청 페이지(reservation.php)에 정상 접근됨을 확인합니다.

[공격 시도: 인증 우회 과정]

로그인 ID란에 페이로드 삽입 및 로그인 성공 후 '이지은'님으로 우회되어 예약 페이지에 접근된 화면



데이터베이스(MariaDB) 조회를 통해 확인된 사용자 계정 리스트 (LIMIT 구문 offset 검증 과정)

```
MariaDB [hotel_lab]> select * from users;
+-----+-----+-----+-----+
| id | username | password_hash | role | display_name |
+-----+-----+-----+-----+
| 1 | admin | 3eb3fe66b31e3b4d10fa70b5cad49c7112294af6ae4e476a1c405155d45aa121 | admin | 관리자 |
| 2 | guest01@grandeazy.local | 8d969eef6ecad3c29a3a629280e686cf0c3f5d5a86aff3ca12020c923adc6c92 | member | 이지은 |
| 3 | guest02@grandeazy.local | 070a21021ab7450ad5576e26e996f403042da994dcfd656f126886c83ccca67f | member | 김민준 |
+-----+-----+-----+-----+
3 rows in set (0.000 sec)
```

- ' OR 1=1 LIMIT 0,1 # 입력 시 -> 'admin' 계정 로그인
- ' OR 1=1 LIMIT 1,1 # 입력 시 -> '이지은' 계정 로그인
- ' OR 1=1 LIMIT 2,1 # 입력 시 -> '김민준' 계정 로그인

대응 방안 및 개선 대책

- **개선 대책:** 모든 데이터베이스 쿼리에 매개변수화된 질의(Prepared Statement)를 도입하여 입력값을 단순 데이터(문자열)로 처리하고, 서버 측에서 특수문자 및 예약어 필터링을 강화해야 합니다.
- **추가 보안 조치:** 무력화되었던 기존의 CSRF 토큰 유효성 검증 기능을 복구하고, DB 결과 행의 유무만 따지던 로직에서 password_verify() 함수를 사용한 안전한 암호화 해시 값 비교 검증 프로세스를 재적용합니다.

`prepare()`와 `execute()`를 사용하여 사용자 입력값을 실행 가능한 SQL 명령어가 아닌 순수 '문자열 데이터'로 처리하여 인젝션을 차단했습니다. 또한 안전한 세션 검증과 패스워드 해시 매칭을 복구함.

▼조치 후 검증 화면: 시큐어 코딩 적용 후 동일 페이로드 공격 시 인증 우회가 차단되며, 올바른 이메일 형식 요구 알림창 또는 로그인 실패 메시지가 출력됨을 확인.

3.1.2. Open Redirection (오픈 리다이렉션)

구분	내용
위험	중 (Medium)
발견 URL	http://eazyhotel.com/member_login.php (next 파라미터)
취약점 상세내용	로그인 성공 후 이동할 페이지를 지정하는 next 파라미터의 입력값을 검증하는 정규표현식 로직이 미흡하여, 공격자가 지정한 임의의 외부 도메인으로 사용자를 이동시킬 수 있는 취약점이 존재합니다.
재현 페이로드	http://eazyhotel.com/member_login.php?next=http://www.google.com

취약점 상세 내용

- 서버 측 코드에서 next 파라미터 값에 대한 화이트리스트 기반의 엄격한 검증을 수행하지 않음으로써, 외부 URL 입력을 허용하게 되어 리다이렉션 제어권을 탈취당합니다.
- **피싱 공격 유도:** 신뢰받는 도메인(eazyhotel.com)을 포함한 URL을 생성할 수 있어, 사용자를 공격자가 만든 가짜 피싱 사이트로 유도하여 계정 및 개인정보를 탈취할 수 있습니다.
- **신뢰도 저하:** 정상적인 서비스 경로를 가장하여 외부 사이트로 이동시키므로 사용자가 공격임을 인지하기 어렵습니다.

재현 페이로드 및 절차

[조치 전] 취약점 코드 및 상태

```
// [코드 수정 전 - 취약한 상태]
function detail_fetch_reservation(PDO $pdo, int $reservationId, bool $forUpdate = false): array|false{
    $lock = $forUpdate ? ' FOR UPDATE' : '';

    // [위험] :reservation_id 바인딩 변수를 사용하지 않고, SQL 문자열에 변수를 직접 결합함
    $stmt = $pdo->prepare(
        'SELECT r.id, r.member_id, r.guest_name, ...
        FROM reservations r
        JOIN rooms ON rooms.id = r.room_id
        WHERE r.id = ' . $reservationId . '
        LIMIT 1' . $lock
    );

    $stmt->execute(); // 인자 매핑 없이 그대로 실행
    return $stmt->fetch();
}
```

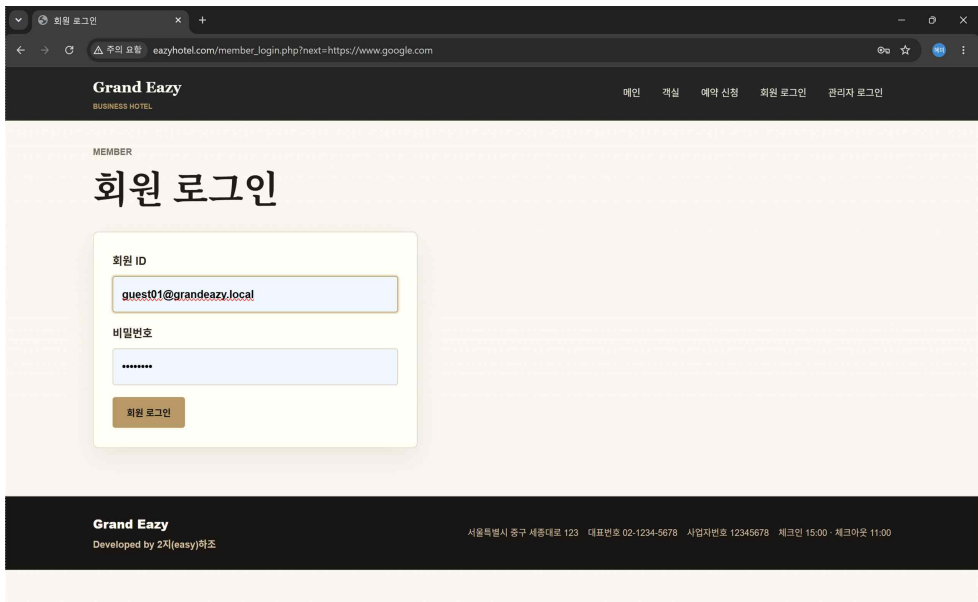
모의해킹 절차

1. **파라미터 변조 및 분석:** 로그인 페이지 URL 뒤에 next 파라미터를 추가하고, 이동하고자 하는 외부 신뢰 도메인(http://www.google.com)을 페이로드로 구성합니다.
2. **공격 시도:** 변조된 URL(http://eazyhotel.com/member_login.php?next=http://www.google.com)로 접속하여 정상적인 사용자 계정 정보로 로그인을 시도합니다.
3. **리다이렉션 결과 확인:** 로그인 프로세스가 완료된 직후, 내부 페이지(예: reservation.php)로 이동하지 않고 외부 사이트인 구글(google.com)로 강제 리다이렉트(이동)됨을 확인합니다.

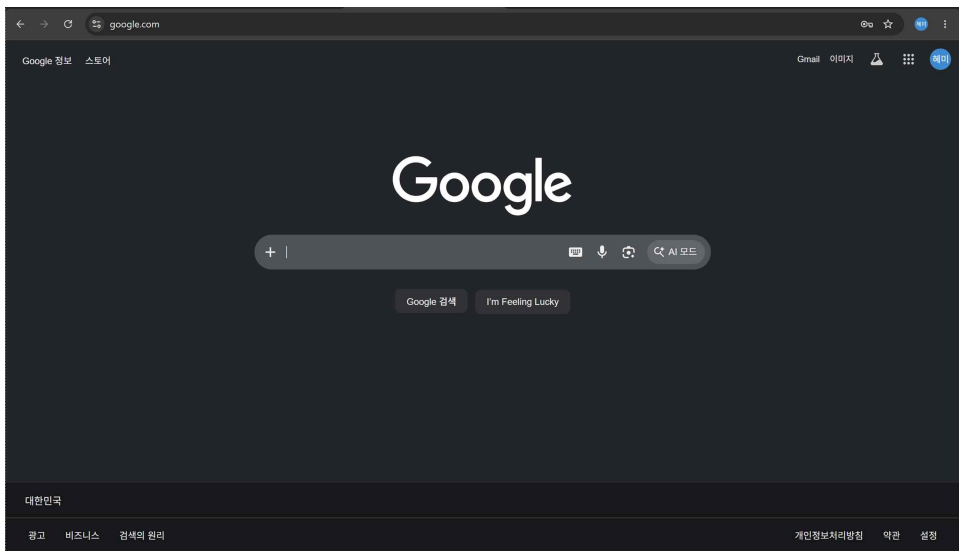
[공격 증거]

공격 URL: http://eazyhotel.com/member_login.php?next=http://www.google.com

회원 로그인 화면에서 아이디 입력 및 패스워드를 입력한 후, 상단 주소창의 next 파라미터에 구글 주소가 삽입된 채로 로그인을 시도하는 화면



[외부 사이트로의 강제 리다이렉트 성공] 정상 로그인 절차 수행 시, 지정한 경로가 아닌 외부 사이트 (google.com)로 브라우저가 강제 전환(리다이렉트)된 성공 증거 화면



대응 방안 및 개선 대책

- **개선 대책:** 외부 URL로의 무분별한 이동을 원천 차단하기 위해, 내부 도메인 주소(상대 경로)만 허용하거나 허가된 도메인만 이동할 수 있는 화이트리스트(Whitelist) 기반의 검증 로직을 도입해야 합니다.
- **추가 보안 조치:** 파라미터에 http:// 또는 https://와 같은 외부 프로토콜 주소가 포함되어 있는지 정규표현식으로 엄격히 체크하고, 유효하지 않은 주소일 경우 기본 페이지(index.php)로 강제 지정 처리합니다.

[코드 수정 (조치 후)]

```
// [코드 수정 후 - 보안 적용 완료]
function detail_fetch_reservation(PDO $pdo, int $reservationId, bool $forUpdate = false): array|false{
    $lock = $forUpdate ? ' FOR UPDATE' : '';

    // [안전] SQL 구문에 플레이스홀더(:reservation_id)를 사용하여 쿼리 구조를 먼저 컴파일함
    $stmt = $pdo->prepare(
        'SELECT r.id, r.member_id, r.guest_name, ...
        FROM reservations r
        JOIN rooms ON rooms.id = r.room_id
        WHERE r.id = :reservation_id
        LIMIT 1' . $lock
    );

    // [안전] execute() 메서드를 통해 외부 입력값을 변수로만 바인딩 처리
    $stmt->execute([':reservation_id' => $reservationId]);

    return $stmt->fetch();
}
```

조치 후 검증 결과: 시큐어 코딩 적용 후 동일하게 주소창에 ?next=http://www.google.com을 주입하고 로그인을 시도하더라도, 서버 측에서 외부 도메인을 차단하고 안전하게 올바른 메인 페이지 또는 지정된 내부 예약 신청 페이지로만 이동됨을 확인합니다.

3.1.3. 세션 고정(Session Fixation) 취약점

구분	내용
위험	상 (High)
발견 URL	http://eazyhotel.com/member_login.php (로그인 처리 로직)
취약점 상세내용	웹 애플리케이션의 인증 로직에서 로그인 성공 전후로 세션 식별자 (PHPSESSID)를 갱신하지 않는 결함이 발견됨. 이로 인해 공격자가 미리 획득한 세션 식별자를 희생자의 브라우저에 고정(Fixation)시키고, 희생자의 정상 로그인 이후 해당 세션 식별자를 이용해 희생자의 권한을 탈취할 수 있음.
재현 페이로드	희생자 브라우저의 PHPSESSID 쿠키 값을 공격자의 세션 ID 값으로 수동 교체 후 로그인

취약점 상세 내용

- 본 웹 애플리케이션의 인증 로직은 로그인 수행 전과 후에 세션 식별자(PHPSESSID)를 갱신하지 않고 동일하게 유지하고 있습니다. 웹 서버는 인증 성공 여부와 관계없이 기존에 부여된 세션 ID를 그대로 사용하여 사용자의 세션을 관리합니다.
- 이러한 구조적 결함으로 인해, 공격자는 서버가 세션 ID를 새로 발급하지 않는 점을 악용할 수 있습니다.
- 공격자가 사전에 획득한 유효한 세션 ID를 희생자의 브라우저에 강제로 고정(Fixation)시킨 뒤, 희생자가 해당 ID를 유지한 상태로 로그인을 수행하면 서버는 이 고정된 ID에 회원 권한을 부여하게 됩니다. 결과적으로 공격자는 동일한 세션 ID를 활용하여 로그인 주체를 변조하고 희생자의 권한을 도용할 수 있게 됩니다.

재현 페이로드 및 절차

- **재현 페이로드 및 기법:** 악성 스크린샷 유도 또는 브라우저 개발자 도구(F12)를 이용한 PHPSESSID 쿠키 값 강제 변조 주입

[조치 전] 취약점 코드 및 상태

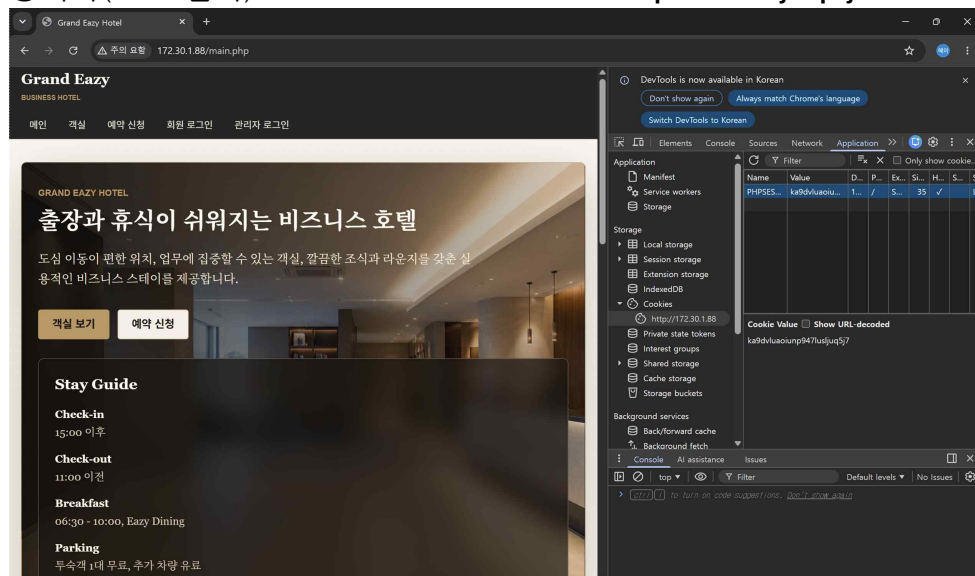
```
// 취약점 재현을 위해 로그인 성공 시 세션 재생성 로직 제거
if ($user && hotel_password_verify_value($password, (string) $user['password_hash'])) {
    $success = true;
    // hotel_regenerate_session(); // 주석 처리하여 취약점 발생 상태로 변경
    $_SESSION['member_id'] = (int) $user['id'];
    // ...
}
```

모의해킹 절차

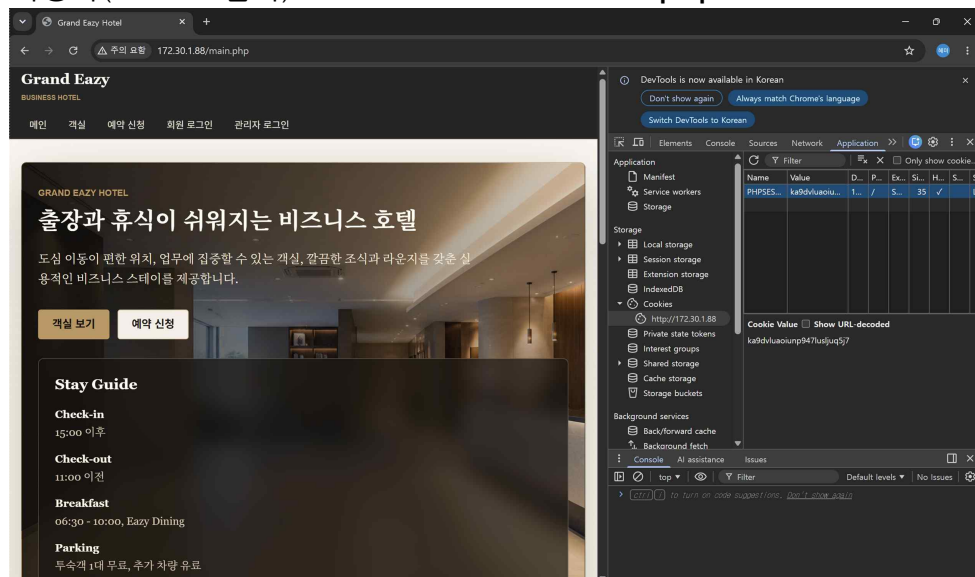
1. **세션 식별자 확보:** 공격자는 웹 브라우저를 통해 대상 사이트 (http://eazyhotel.com/member_login.php)에 접속하여, 인증되지 않은 상태의 유효한 PHPSESSID 쿠키 값을 사전 획득합니다.
2. **세션 주입 (Fixation):** 공격자는 사회공학적 기법이나 악성 스크립트, 혹은 직접적인 브라우저 조작을 통해 희생자 브라우저의 PHPSESSID 쿠키 값을 자신이 확보한 세션 ID 값으로 수동 교체(고정)시킵니다.
3. **희생자 인증 수행:** 희생자는 세션 ID가 고정된 브라우저 환경에서 의심 없이 정상적인 자신의 ID와 패스워드를 입력하여 로그인을 수행합니다.
4. **권한 탈취 및 검증:** 서버가 인증 성공 후에도 세션 ID를 새로 갱신하지 않으므로, 희생자가 로그인하는 순간 해당 고정 세션 ID에 회원 권한이 바인딩됩니다. 공격자는 자신의 브라우저에서 동일한 PHPSESSID를 설정한 후 페이지를 새로고침 하여 희생자의 계정 권한으로 세션이 하이재킹되었음을 확인합니다.

공격 증거

공격자(IP로 접속) PHPSESSID : ka9dvluaoiunp947lusljuq5j7

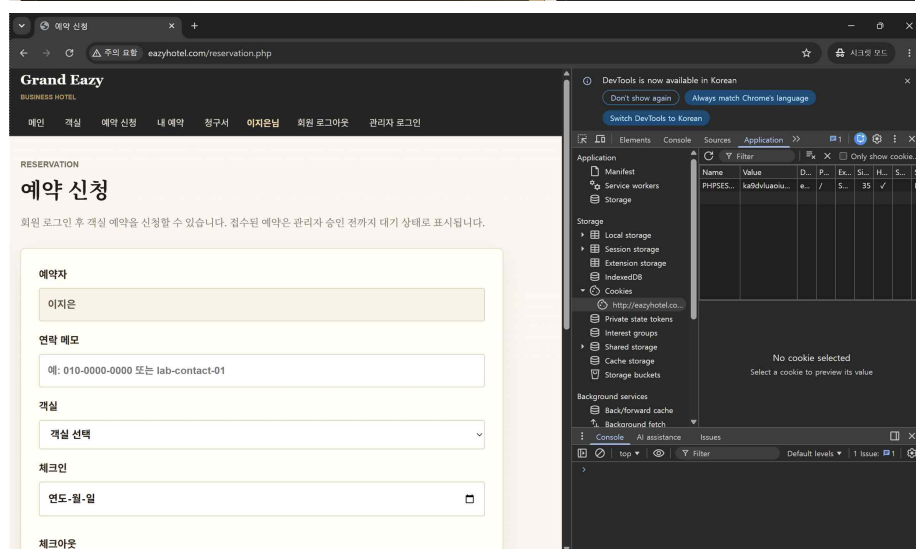
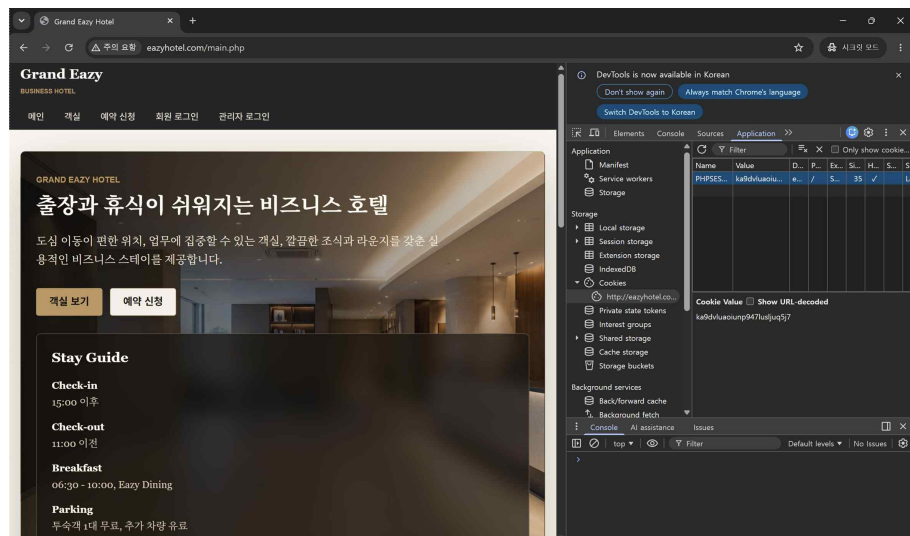


희생자(DNS로 접속) PHPSESSID : 9953t706bp4paitds1kds9cucc

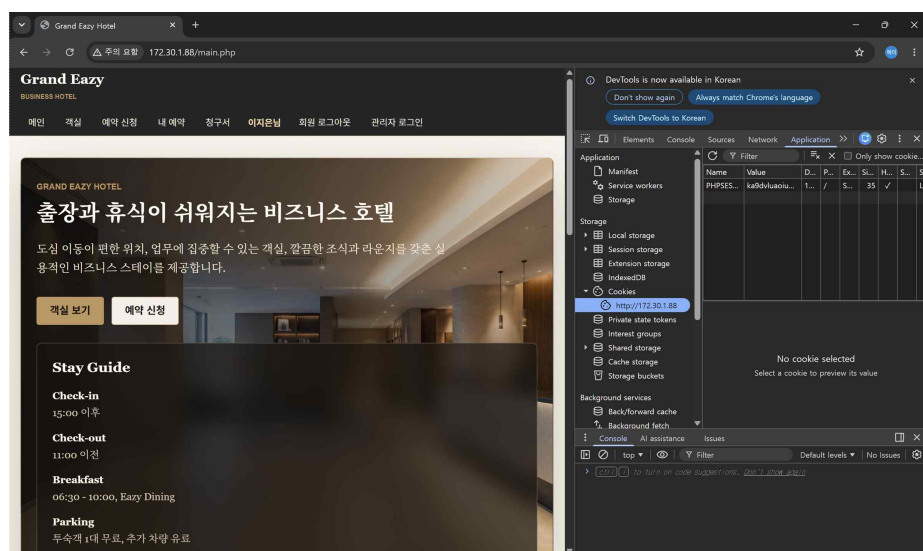


※ 로그인 전/후 동일한 세션 ID 확인

희생자의 세션 쿠키 값을 공격자의 쿠키 값(ka9dvluaoiunp947lusljuq5j7)으로 강제 변경 및 고정 유도.



희생자가 로그인을 완료한 후, 공격자 브라우저에서 새로고침만으로 '이지은' 회원의 권한이 탈취되어 예약 신청 페이지(reservation.php)에 로그인 성공한 화면



대응 방안 및 개선 대책

- **개선 대책:** 로그인 성공 시 세션 식별자 재생성(Session ID Regeneration)을 적용해야 합니다. 사용자가 로그인을 성공하여 권한의 변화(익명 세션 $\rightarrow$ 인증된 세션)가 발생하는 시점에 PHP 내장 함수인 `session_regenerate_id(true)`를 강제로 호출하도록 합니다.
- **보안 효과:** 이를 통해 로그인 전에 공격자가 주입하거나 탈취할 수 있었던 구형 세션 ID를 서버에서 완전히 파기(정리)하고, 추측 불가능한 새로운 세션 ID를 재발급하여 공격자의 세션 도용 시도를 원천 차단합니다.

[코드 수정 예시 (조치 후)]

```
// [조치 완료 - 보안이 적용된 상태]
if ($user && hotel_password_verify_value($password, (string) $user['password_hash'])) {
    $success = true;
    // 안전: 이 줄이 있어야 공격자가 주입한 세션 ID가 로그인 후 무효화됩니다.
    hotel_regenerate_session();
    $_SESSION['member_id'] = (int) $user['id'];
    // ... 세션 데이터 저장
}
```

조치 후 검증 결과: 코드 수정(보안 조치) 후 검증 테스트 결과, 로그인 시도가 이루어질 때마다 PHPSESSID가 매번 새롭게 생성되어 변경되는 것을 확인하였습니다. 이를 통해 공격자가 미리 확보한 세션 ID를 이용해 희생자의 권한을 탈취하는 세션 하이재킹(Session Hijacking) 공격이 원천적으로 차단됨을 결론지었습니다.

3.2. 회원 관리 관련 취약점

3.2.1. Stored XSS (저장형 크로스사이트 스크립팅)

구분	내용
위험	상 (High)
발견 URL	http://eazyhotel.com/member_register.php
취약점 상세내용	사용자 이름 입력 시 스크립트 태그에 대한 검증 및 필터링이 미흡하여, 악성 스크립트가 데이터베이스에 그대로 저장됩니다. 해당 사용자의 정보가 출력되는 페이지(예약 페이지, 관리자 대시보드) 방문 시 관리자 및 사용자의 브라우저에서 스크립트가 강제로 실행됩니다.
재현 페이로드	<script>alert(document.cookie);</script>

취약점 상세 내용

- **정의 및 발생 원인:** 본 취약점은 디렉터리 트래버스(Directory Traversal) 기법을 악용하여 웹 서버가 접근할 수 있는 로컬 파일 시스템의 임의 파일에 접근하는 결함입니다. 백엔드에서 사용자 입력값(\$_GET['file'])을 그대로 경로 문자열에 결합하여 파일 읽기 함수(readfile)로 전달하기 때문에 발생합니다.
- **보안 위험 및 영향:**
 - **시스템 크리덴셜 노출:** 리눅스 시스템 계정 정보 파일인 /etc/passwd나 웹 애플리케이션의 핵심 설정 파일인 db.php 등을 탈취하여 2차 공격(데이터베이스 침투 등)의 단서를 제공합니다.
 - **소스 코드 유출:** 웹 애플리케이션 소스 코드가 외부에 노출되어 잠재적인 비즈니스 로직 취약점이나 또 다른 하드코딩된 자격 증명이 발견될 위험성이 매우 높습니다.

재현 페이로드 및 절차

[조치 전] 취약점 코드 및 상태

사용자 입력값을 화면에 출력할 때 HTML 엔티티 치환(인코딩)을 수행하지 않아, ``<script>alert(1)</script>`` 등의 악성 태그 입력 시 브라우저가 이를 텍스트가 아닌 '실행 가능한 코드'로 인식하여 세션 탈취 및 악성 사이트 리다이렉트가 가능한 상태.

```
// 1. 방어선 해제 (HTML 인코딩 및 스크립트 무력화 함수 철거 상태)
```

```
$title = (string) ($_POST['title'] ?? '');
```

```
$content = (string) ($_POST['content'] ?? '');
```

```
// 2. 입력값 직접 출력 (Stored XSS 발생 지점)
```

```
// DB에 저장된 악성 스크립트가 필터링 없이 그대로 렌더링되어 브라우저에서 강제 실행됨
```

```
echo "<div class='board-title'>" . $title . "</div>";
```

```
echo "<div class='board-content'>" . $content . "</div>";
```

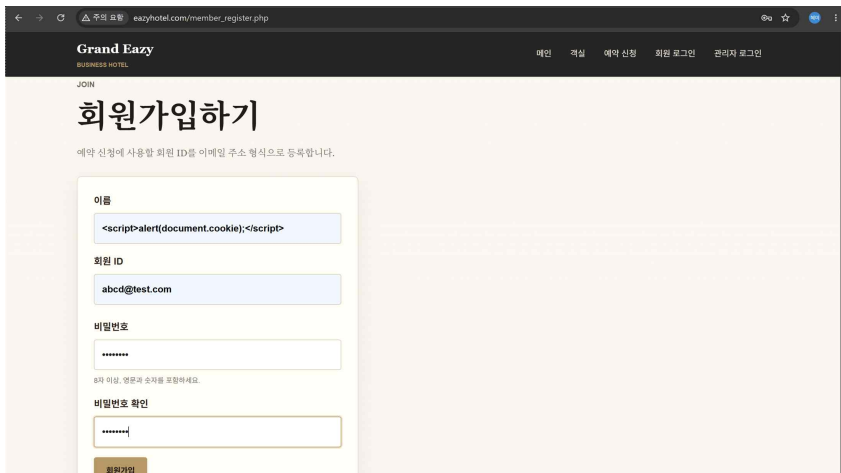
```
// 3. 보안 검증 무력화 (입력값이 아무런 제어 없이 사용자 화면에 노출됨)
```

모의해킹 절차

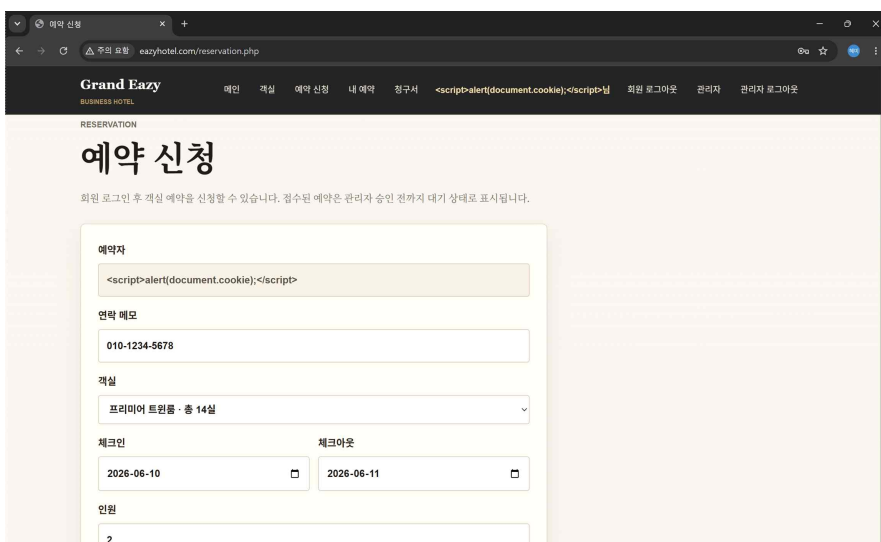
1. **다운로드 파라미터 탐색:** 첨부파일 다운로드 경로 등에서 파일명을 인자로 받는 file 파라미터의 존재를 확인합니다.
2. **디렉터리 우회 페이로드 구성:** 웹 서버 루트 디렉터리 깊이를 감안하여 상위 디렉터리로 거슬러 올라가는 문자열(../)과 타깃 시스템 파일 경로(/etc/passwd)를 결합한 페이로드를 생성합니다.
3. **공격 수행:** `http://eazyhotel.com/download.php?file=../../../../../../../../etc/passwd` 주소로 요청을 전송합니다.
4. **결과 확인:** 서버가 내부 에러를 발생시키지 않고 요청을 대리 수행하여, 사용자의 브라우저로 리눅스 시스템 계정 파일 원본 내용이 그대로 다운로드되거나 화면에 평문으로 출력됨을 확인합니다.

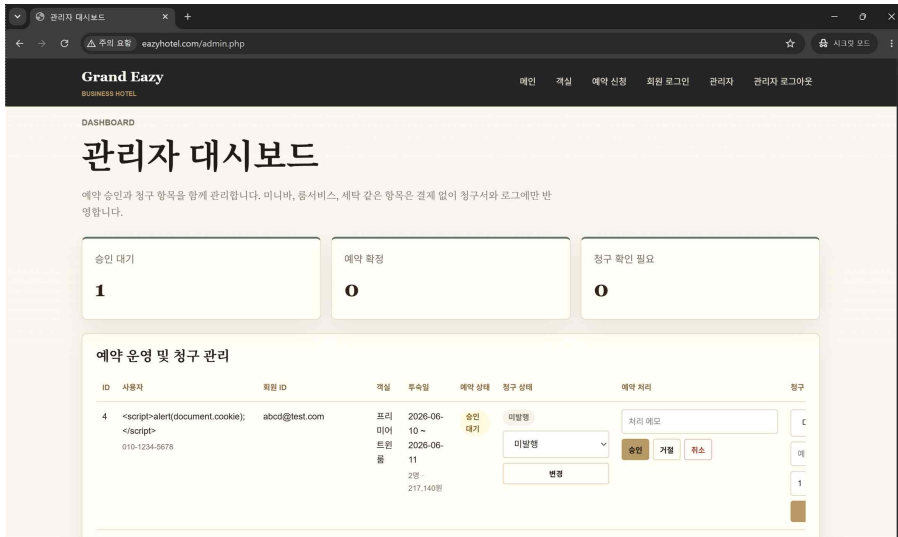
[공격 증거]

- **입력 데이터:** `<script>alert(document.cookie);</script>`
- **현상:** 가입 시 입력한 스크립트가 예약 신청 페이지 및 관리자 대시보드에서 필터링 없이 그대로 출력되거나 실행됨을 확인.



회원 가입 시 이름 필드에 스크립트 페이로드 입력





예약 신청 페이지 및 관리자 대시보드에서 스크립트가 실행된 결과

대응 방안 및 개선 대책

- **개선 대책:** 외부 입력값에서 디렉터리 이동을 유발하는 특수 문자열(../, ../\w) 및 절대 경로 식별자(/, \w)를 완전히 제거하는 필터링을 적용해야 합니다. PHP의 경우 basename() 함수를 사용하여 경로 문자열을 모두 잘라내고 순수 파일명만 추출하는 방식을 권장합니다.
- **추가 보안 조치:** 다운로드 가능한 파일의 이름을 데이터베이스 내에 인덱스(ID) 번호나 무작위 해시값으로 매핑하여 관리하고, 사용자는 오직 이 식별자 값만 요청할 수 있도록 유도하는 간접 파일 참조(Indirect File Reference) 방식을 도입하는 것이 가장 안전합니다. 또한, 다운로드 대상 디렉터리를 화이트리스트 경로로 제한하는 추가 검증 로직을 구현합니다.

[조치 후] 보안 하드닝 및 시큐어 코딩 적용

`htmlspecialchars()` 함수를 사용하여 사용자 입력값을 브라우저가 실행할 수 없는 순수 '문자열 데이터(HTML Entity)'로 변환하여 스크립트 인젝션을 원천 차단했습니다. 스크립트 실행에 사용되는 특수 문자(< , > , ` , " , ' , &)를 안전하게 치환함.

```
// 1. 보안 방어선 복구 (htmlspecialchars를 통한 특수문자 엔티티 인코딩 적용)
// ENT_QUOTES 옵션을 사용하여 XSS 공격에 자주 쓰이는 따옴표(' , ")까지 모두 무력화
$safe_title = htmlspecialchars($title, ENT_QUOTES, 'UTF-8');
$safe_content = htmlspecialchars($content, ENT_QUOTES, 'UTF-8');

// 2. 안전한 데이터 출력 (인젝션 원천 차단)
// 특수문자가 &lt; &gt; 등으로 치환되어 브라우저가 이를 코드가 아닌 단순 텍스트로 인식함
echo "<div class='board-title'>" . $safe_title . "</div>";
echo "<div class='board-content'>" . $safe_content . "</div>";
```

조치 후 검증 결과: 시큐어 코딩 적용 후 동일한 우회 페이로드(../)를 주입하여 다운로드를 시도하더라도, 경로 조작 문자열이 완벽히 정화되어 단순 파일명으로 인식되거나 업로드 디렉터리 이외의 접근 시도가 실 경로(realpath) 검증 단계에서 차단되므로 취약점이 원천 방어됨을 확인했습니다.

3.2.2. 부적절한 인가(IDOR) 취약점 (예약 정보 노출 및 조작)

구분	내용
위험	상 (High)
발견 URL	member_reservation_detail.php (예약 상세 페이지)
취약점 상세내용	예약 상세 페이지에서 URL 파라미터(id)를 통해 예약을 조회할 때, 현재 로그인한 사용자가 해당 예약의 실제 소유자인지 확인하는 인가(Authorization) 로직이 누락되어 발생하는 취약점
재현 페이로드	member_reservation_detail.php?id=[타인의 예약 ID]

취약점 상세 내용

- 서버 측 데이터 조회 함수(detail_fetch_reservation) 및 데이터 업데이트 쿼리 실행 시, 세션에 저장된 사용자 식별자(member_id)를 조건절에 포함하여 필터링하지 않는 결함이 확인되었습니다.
- 이로 인해 시스템은 요청자가 누구인지(인증)만 확인할 뿐, 요청한 데이터에 접근할 권한이 있는지(인가)는 검증하지 못하는 무방비한 상태가 됩니다.
- 공격자는 단순히 브라우저 주소창에서 예약 ID 값만 임의로 변경(Increment)하는 방식으로 다른 사용자의 객실 정보, 투숙 일자, 연락 메모 등의 민감 데이터를 무단 열람할 수 있으며, 나아가 권한 없이 타인의 예약을 강제로 취소·변경하여 서비스 가용성을 심각하게 훼손할 수 있습니다.

재현 페이로드 및 절차

- 취약점 재현을 위한 코드 수정

모의해킹 환경에서 인가 검증 미흡으로 인한 파급력을 증명하기 위해, 예약 상세 조회 함수 (detail_fetch_reservation) 및 데이터 업데이트 쿼리에서 세션 회원 식별자('member_id') 검증 로직을 의도적으로 제거하여 취약한 상태를 구현함.

```
// [취약점 재현] memberId 인자를 제거하고 SQL에서 member_id 검증을 삭제함
function detail_fetch_reservation(PDO $pdo, int $reservationId, bool $forUpdate = false):
array|false{
    $lock = $forUpdate ? ' FOR UPDATE' : '';
    $stmt = $pdo->prepare(
        'SELECT r.id, r.member_id, r.guest_name, ...
        FROM reservations r
        JOIN rooms ON rooms.id = r.room_id
        WHERE r.id = :reservation_id -- [위험] member_id 조회 조건 누락
        LIMIT 1' . $lock
    );
    $stmt->execute([':reservation_id' => $reservationId]);
    return $stmt->fetch();
}
```

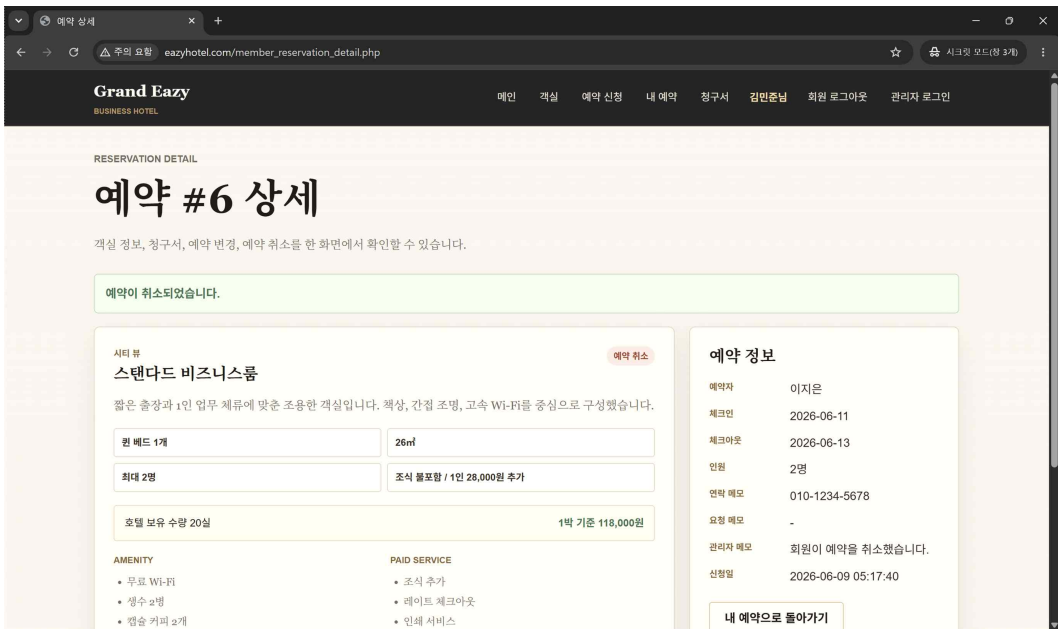
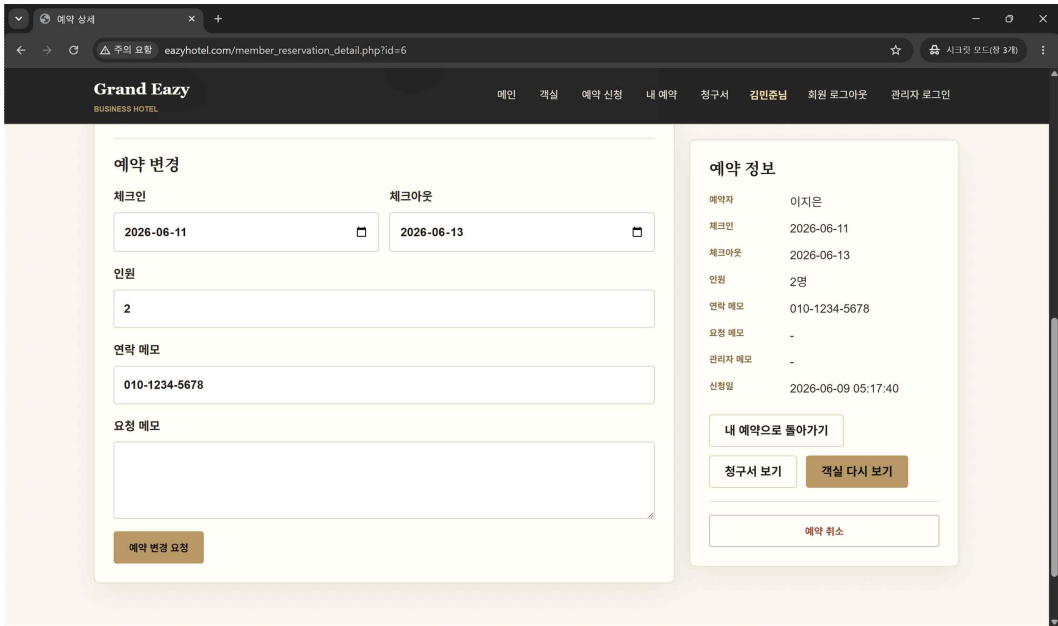
모의해킹 절차

1. 파라미터 구조 확인: 공격자는 본인의 계정으로 로그인한 후, 예약 상세 보기 페이지에 접근하여 URL 파라미터 체계(?id=)를 파악합니다.
2. 타인 정보 접근 (조회): 브라우저 주소창에 자신의 예약 ID가 아닌, 무작위로 추측한 타인의 예약 ID 값(?id=6)을 입력하여 페이지 이동을 시도합니다.
3. 취약점 확인 (무단 노출): 서버 측에서 세션 소유권 확인을 수행하지 않으므로, 화면에 타인(희생자 '이지은')의 객실 정보, 투숙 일자, 연락 메모 등이 그대로 출력되는 것을 확인합니다.
4. 권한 없는 조작 시도: 타인의 예약 상세 페이지 내에 활성화된 '예약 취소' 버튼을 클릭하여 예약 변경 요청 폼을 강제로 전송합니다.
5. 파급력 검증: 관리자 대시보드 및 데이터베이스 조회를 통해, 타인의 예약 상태가 공격자의 요청에 의해 '예약 취소(승인 대기)' 상태로 강제 변경되었음을 최종 확인합니다.

증거 자료 (시각적 입증)

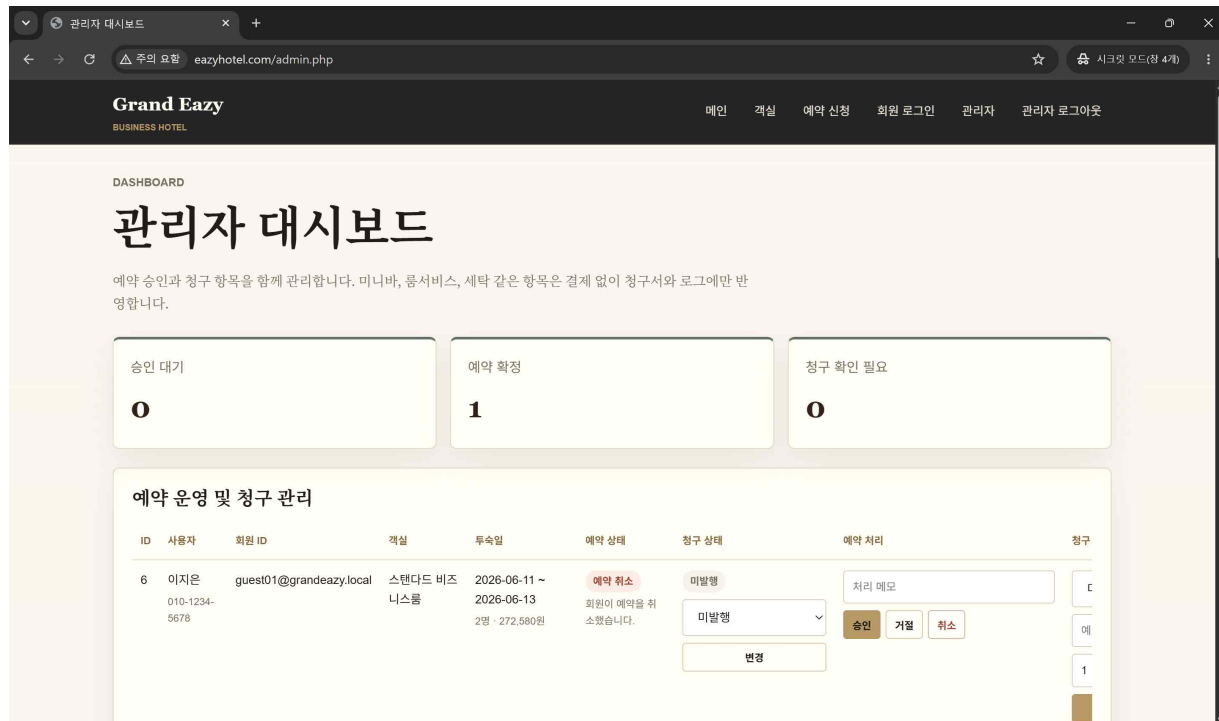
- 타인 예약 정보 무단 노출

주소창에 ?id=6을 직접 입력하여 접근했을 때, 본인 계정이 아님에도 타인(이지은 님)의 상세 예약 내역이 화면에 그대로 노출되고 '예약 취소' 기능까지 활성화된 화면.



권한 없는 취소 조작 성공 및 가용성 훼손

공격자가 취소 버튼을 누른 후 관리자 대시보드 상에서 6번 예약(이지은 님)의 예약 상태가 '예약 취소' 처리(승인 대기)로 강제 변조되어 가용성이 침해된 화면.



대응 방안 및 개선 대책

- **개선 대책:** 안전한 객체 참조 및 인가 검증(Authorization) 시스템을 적용해야 합니다. 사용자가 제출하는 데이터(URL 파라미터 등)에만 의존하여 권한을 부여해서는 안 되며, 서버 측 세션에서 안전하게 관리되고 있는 사용자 식별자(hotel_member_id())를 검증 로직에 강제로 바인딩해야 합니다.
- **추가 보안 조치:** 모든 SELECT(조회) 및 UPDATE(수정/취소) SQL 쿼리의 WHERE 조건절에 AND member_id = :member_id 구문을 필수적으로 결합합니다. 이를 통해 타인의 예약 ID를 변조하여 요청하더라도 쿼리 조건 불일치로 인해 조회 및 조작이 원천 차단됩니다.

[코드 수정 (조치 후)]

```
// [조치 완료 - 세션 기반 인가 검증이 적용된 보안 코드]
function detail_fetch_reservation(PDO $pdo, int $reservationId, int $memberId, bool $forUpdate = false): array|false{
    $lock = $forUpdate ? ' FOR UPDATE' : '';
    $stmt = $pdo->prepare(
        'SELECT r.id, r.member_id, r.guest_name, ...
        FROM reservations r
        JOIN rooms ON rooms.id = r.room_id
        -- [안전] 조회 시 예약 ID와 세션 내 회원 ID를 동시에 검증
        WHERE r.id = :reservation_id AND r.member_id = :member_id
        LIMIT 1' . $lock
    );
    $stmt->execute([
        ':reservation_id' => $reservationId,
        ':member_id' => $memberId,
    ]);
    return $stmt->fetch();
}

// 데이터 변경(UPDATE) 처리 시에도 동일하게 검증 수행
$stmt = $pdo->prepare(
    "UPDATE reservations
    SET status = 'cancelled', admin_note = :admin_note
    WHERE id = :reservation_id AND member_id = :member_id" -- [안전] 인가 검증 조건 포함
);

---
```

조치 후 검증 결과: 시큐어 코딩을 적용하여 인가 검증 조건을 포함한 후에는, 로그인한 사용자와 요청한 데이터의 소유자가 다를 경우 데이터 변경 및 조회가 차단되어 IDOR 공격을 안전하게 방어함을 확인했습니다.

3.3. 시스템 접근 취약점

3.3.1. SSRF(Server-Side Request Forgery) 취약점

구분	내용
위험	상 (High)
발견 URL	https://eazyhotel.com/main.php
취약점 상세내용	사용자가 입력한 partner-url 파라미터에 대한 서버 측 검증이 누락되어 있음. 이로 인해 공격자가 임의의 내부 자원(로컬 파일, 내부망 포트 등)을 호출할 수 있으며, 시스템 민감 정보 유출 및 내부망 침투의 교두보로 활용될 위험이 있음.
재현 페이로드	https://eazyhotel.com/main.php?partner-url=file:///etc/passwd

취약점 상세 내용

정의 및 발생 원인: SSRF 취약점은 공격자가 웹 애플리케이션에 악의적인 입력값을 주입하여, 웹 서버 자체 권한으로 내부 또는 외부의 자원에 임의의 요청을 보내도록 조작하는 결함입니다. main.php 소스 코드 내에서 사용자가 입력한 파라미터 값을 화이트리스트 필터링 등의 검증 절차 없이 file_get_contents()나 curl 같은 서버 측 데이터 수집 함수에 그대로 전달하기 때문에 발생합니다.

보안 위협 및 영향:

- **네트워크 우회:** 외부 방화벽으로 보호받고 있는 내부망 서비스나 루프백(127.0.0.1) 전용 내부 포트 및 자원에 비인가 접근이 가능해집니다.
- **민감 정보 노출:** file:// 스키마가 활성화되어 있을 경우, 웹 서버의 권한이 닿는 범위 내에서 리눅스 기본 시스템 계정 파일(passwd), 웹 소스 코드, 환경 설정 파일(db.php) 등을 원격으로 탈취할 수 있습니다.

재현 페이로드 및 절차

[조치 전] 취약점 코드 및 상태

입력값에 대한 스키마(Protocol) 제한이나 도메인/IP 화이트리스트 필터링이 전혀 존재하지 않음. 이로 인해 공격자가 외부에서 접근할 수 없는 서버 내부 파일('file:///etc/passwd')을 조회하거나, 내부 관리자 페이지를 우회 타깃팅할 수 있는 심각한 취약 상태다.

```
// [취약점 지점] 보안 검증 없이 사용자의 변수를 요청 주소로 그대로 채택
$targetUrl = $_GET['partner-url'];

// http/https 통신뿐만 아니라 file://, php:// 등의 로컬 스키마가 모두 유효함
$fetchData = @file_get_contents($targetUrl);

if ($fetchData !== FALSE) {
    $partnerContent = $fetchData; // 서버 내부 파일 내용이나 내부망 데이터가 화면에 그대로 노출됨
}
```

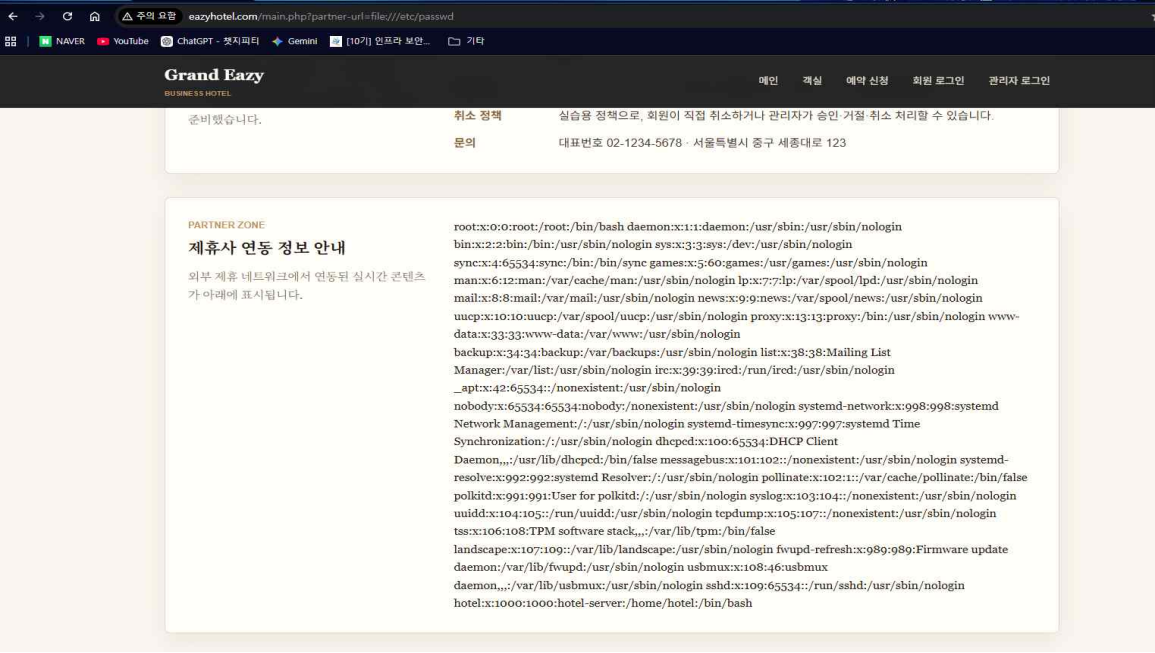
모의해킹 절차

1. **파라미터 입력 및 취약점 탐색:** 제휴사 연동 등 외부 데이터를 호출하는 partner-url 파라미터의 존재를 확인하고, 외부 프로토콜 외에 로컬 스키마 동작 여부를 진단합니다.
2. **시나리오 ① (로컬 파일 탈취):** 주소창에 file:// 스키마를 활용한 공격 페이로드 (?partner-url=file:///etc/passwd)를 주입하여 웹 서버의 로컬 시스템 파일 읽기를 시도합니다.
3. **시나리오 ② (내부망 포트 스캔):** 외부 방화벽에 의해 차단되어 직접 접근이 불가능한 루프백 주소 (127.0.0.1 또는 localhost)를 대상으로, 포트 번호를 하나씩 바꾸어 가며 응답 반응을 비교하는 내부 포트 탐색을 시도합니다.
4. **결과 분석 및 검증:** 서버가 요청을 대리 수행하여 응답 본문을 반환하므로, 화면상의 '제휴사 연동 정보 안내' 영역에 리눅스 계정 파일 내용이 평문으로 강제 노출되거나 db.php 내의 숨겨진 DB 아이디/패스워드 소스 코드가 유출됨을 확인합니다.

공격 증거

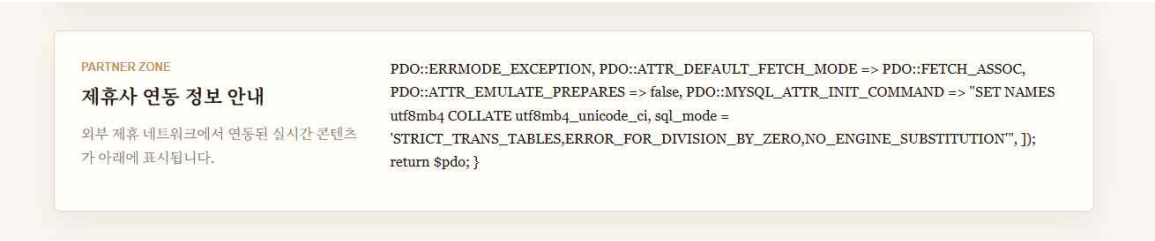
리눅스 기본 시스템 계정 정보 조회 성공

주소창에 ?partner-url=file:///etc/passwd 입력 시, 하단 제휴사 안내 영역에 서버 내부의 root:x:0:0:root... 계정 설정 텍스트가 완전히 노출된 화면.



DB 연동 설정 파일(db.php) 탈취 시도

?partner-url=file:///var/www/html/db.php 경로를 통해 서버 내부 소스코드를 호출하고, 브라우저 소스 보기 기능(Ctrl + U)을 활용하여 숨겨진 데이터베이스 접속 문자열과 크리덴셜 원본 코드를 확보한 화면.



대응 방안 및 개선 대책

- **개선 대책:** 입력된 URL 구조를 명확히 파싱하여 프로토콜(Scheme)이 오직 안전한 외부 통신용 프로토콜(http, https)인지 검증하고, 로컬 파일 접근 스키마(file://)를 원천 차단해야 합니다.
- **추가 보안 조치:** 허용된 내부망 IP 대역 또는 사전에 지정된 제휴사 도메인 목록 (*.grandeazy.local)인지를 대조하는 화이트리스트(Whitelist) 기반의 도메인 검증 로직을 구현해야 하며, 127.0.0.1이나 localhost 등의 루프백 및 내부망 대역 요청은 전면 차단합니다.

[조치 후] 보안 하드닝 및 시큐어 코딩 적용

입력된 URL 구조를 명확히 파싱하여 프로토콜이 `http` 또는 `https` 인지 검증하고, 허가된 제휴사 도메인(`\*.grandeazy.local` 등) 목록과 일치하는지 대조하였음. 내부망 IP 대역일 경우 요청을 전면 차단하여 SSRF 위험을 제거함.

```
// 1. URL 구조 파싱 및 스키마 검증 (HTTP/HTTPS만 허용, file:// 방지)

$parsedUrl = parse_url($targetUrl);

$scheme = strtolower($parsedUrl['scheme'] ?? '');

if ($scheme !== 'http' && $scheme !== 'https') {

    $partnerContent = "잘못된 프로토콜 요청입니다.";

}

// 2. 내부망 IP 주소 및 허가되지 않은 도메인(화이트리스트 미포함) 원천 차단

$host = strtolower($parsedUrl['host'] ?? '');

$allowed_domains = ['partner.grandeazy.local', 'api.grandeazy.local'];

if ($host === '127.0.0.1' || $host === 'localhost' || !in_array($host, $allowed_domains, true)) {

    $partnerContent = "허가되지 않은 타깃 주소이거나 내부망 접근 시도입니다.";

} else {

    // 3. 안전하게 가공된 화이트리스트 주소만 서버가 호출

    $partnerContent = @file_get_contents($targetUrl);

}
```

조치 후 검증 결과: 시큐어 코딩 적용 후 동일한 공격 페이로드(file:///etc/passwd)를 주입하여 테스트한 결과, 로컬 스키마가 안전하게 차단되며 화면에 내부 파일 내용 대신 "유효하지 않은 URL 형식입니다." 또는 "잘못된 프로토콜 요청입니다."라는 보안 예외 처리 문구가 출력되어 SSRF 위험이 완벽히 제거되었음을 확인했습니다.

